Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	**Date:**	**Enrolment No : 92301733009**

Aim: To study different word embedding for representation of textual data in vectorized form

IDE: Google Colab

Theory:

Word Embedding is an approach for representing words and documents. Word Embedding or Word Vector is a numeric vector input that represents a word in a lower-dimensional space. It allows words with similar meanings to have a similar representation.

Word Embeddings are a method of extracting features out of text so that we can input those features into a machine learning model to work with text data. They try to preserve syntactical and semantic information. The methods such as Bag of Words (BOW), CountVectorizer and TFIDF rely on the word count in a sentence but do not save any syntactical or semantic information. In these algorithms, the size of the vector is the number of elements in the vocabulary. We can get a sparse matrix if most of the elements are zero. Large input vectors will mean a huge number of weights which will result in high computation required for training. Word Embeddings give a solution to these problems.

Need for Word Embedding?

- To reduce dimensionality
- To use a word to predict the words around it.
- Inter-word semantics must be captured.

How are Word Embeddings used?

- They are used as input to machine learning models.
  Take the words —-> Give their numeric representation —-> Use in training or inference.
- To represent or visualize any underlying patterns of usage in the corpus that was used to train them.

1. Bag of Words

Bag of Words (BOW) is an algorithm that counts how many times a word appears in a document. Those word

counts allow us to compare documents and gauge their similarities for applications like search, document classification, and topic modeling.

2. Tf-Idf

Tf-Idf is shorthand for term frequency-inverse document frequency. So, two things: term frequency and inverse document frequency. Term frequency (TF) is basically the output of the BoW model. For a specific document, it determines how important a word is by looking at how frequently it appears in the document. Term frequency measures the importance of the word. If a word appears a lot of times, then the word must be important. For example, if our document is "I am a cat lover. I have a cat named Steve. I feed a cat outside my room regularly," we see that the words with the highest frequency are I, a, and cat. This agrees with our intuition that high term frequency = higher importance.

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	**Date:**	**Enrolment No : 92301733009**

TF(t) = (Number of times term t appears in a document) / (Total number of terms in the document)

IDF used to calculate the weight of rare words across all documents. The words that occur rarely in the corpus have a high IDF score. However, it is known that certain terms, such as “I”, “a” may appear a lot of times but have little importance. Thus we need to weigh down the frequent terms while scaling up the rare ones

IDF(t) = log_e(Total number of documents / Number of documents with term t in it)

Pre Lab Exercise:

[8] ✓ 0s

```
# Sample text data
corpus = [
    "I love machine cyber security",
    "Cybersecurity is very nice",
    "I love write scripts"
]
```

[9] ✓ 0s

```
'''# Sample text data
corpus = [
    "I love machine learning",
    "Machine learning is amazing",
    "I love coding in python"
]
'''
# ------------------------------
# Bag of Words using CountVectorizer
# ------------------------------
from sklearn.feature_extraction.text import CountVectorizer

# Create CountVectorizer object
vectorizer = CountVectorizer()
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	**Date:**	**Enrolment No : 92301733009**

```
# Create CountVectorizer object
vectorizer = CountVectorizer()

# Fit and transform the corpus
bow_matrix = vectorizer.fit_transform(corpus)

# Convert sparse matrix to array
bow_array = bow_matrix.toarray()

# Get feature (word) names
bow_features = vectorizer.get_feature_names_out()

# Display results
print("Vocabulary:", bow_features)
print("\nBag of Words Matrix:\n", bow_array)


# ------------------------------
# Term Frequency (TF) using TfidfVectorizer (without IDF)
# ------------------------------
from sklearn.feature_extraction.text import TfidfVectorizer

# Initialize TF vectorizer (IDF disabled)
tf_vectorizer = TfidfVectorizer(use_idf=False, norm='l1')
```

```
# Initialize TF vectorizer (IDF disabled)
tf_vectorizer = TfidfVectorizer(use_idf=False, norm='l1')

# Fit and transform the corpus
tf_matrix = tf_vectorizer.fit_transform(corpus)

# Convert to array
tf_array = tf_matrix.toarray()

# Get feature names
tf_features = tf_vectorizer.get_feature_names_out()

# Display results
print("\nVocabulary:", tf_features)
print("\nTF Matrix:\n", tf_array)
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	**Date:**	**Enrolment No : 92301733009**

```
Vocabulary: ['cyber' 'cybersecurity' 'is' 'love' 'machine' 'nice' 'scripts' 'security'
 'very' 'write']

Bag of Words Matrix:
 [[1 0 0 1 1 0 0 1 0 0]
 [0 1 1 0 0 1 0 0 1 0]
 [0 0 0 1 0 0 1 0 0 1]]

Vocabulary: ['cyber' 'cybersecurity' 'is' 'love' 'machine' 'nice' 'scripts' 'security'
 'very' 'write']

TF Matrix:
 [[0.25       0.         0.         0.25       0.25       0.
  0.         0.25       0.         0.        ]
 [0.         0.25       0.25       0.         0.         0.25
  0.         0.         0.25       0.        ]
 [0.         0.         0.         0.33333333 0.         0.
  0.33333333 0.         0.         0.33333333]]
```

Program (Code):

sentences = ["I feel proud to be a citizen of Mozambique because of its rich culture, traditions, and history.", "It is known for its festivals, traditions, and historical monuments.",

"India is the world’s largest democracy.",

"Technology makes our work easier and faster.", "It helps us communicate and learn better.", "Modern life depends heavily on technology.",

"Sports help people stay healthy and active.",

"They teach teamwork, discipline, and confidence.", "in my free time i would love to play cricket."

]

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	**Date:**	**Enrolment No : 92301733009**

```
import numpy as np
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer #TF-Bow
from sklearn.feature_extraction.text import TfidfVectorizer #TFIDF
from sklearn.metrics.pairwise import cosine_similarity
#Embeddings using TF_Bow approach
tfidf_vectorizer = TfidfVectorizer()
tfidf_matrix = tfidf_vectorizer.fit_transform(sentences)
tfidf_matrix
tfidf_similarity=cosine_similarity(tfidf_matrix)
tfidf_similarity
```

Results

```
array([[1.        , 0.15167348, 0.        , 0.02814919, 0.02814919,
        0.        , 0.0274983 , 0.02999385, 0.06156273],
       [0.15167348, 1.        , 0.10471098, 0.03876979, 0.14108832,
        0.        , 0.03787332, 0.04131044, 0.        ],
       [0.        , 0.10471098, 1.        , 0.        , 0.        ,
        0.        , 0.        , 0.        , 0.        ],
       [0.02814919, 0.03876979, 0.        , 1.        , 0.04517405,
        0.12200786, 0.0441295 , 0.04813438, 0.        ],
       [0.02814919, 0.14108832, 0.        , 0.04517405, 1.        ,
        0.        , 0.0441295 , 0.04813438, 0.        ],
       [0.        , 0.        , 0.        , 0.12200786, 0.        ,
        1.        , 0.        , 0.        , 0.        ],
       [0.0274983 , 0.03787332, 0.        , 0.0441295 , 0.0441295 ,
        0.        , 1.        , 0.04702137, 0.        ],
       [0.02999385, 0.04131044, 0.        , 0.04813438, 0.04813438,
        0.        , 0.04702137, 1.        , 0.        ],
       [0.06156273, 0.        , 0.        , 0.        , 0.        ,
        0.        , 0.        , 0.        , 1.        ]])
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	**Date:**	**Enrolment No : 92301733009**

Observation:

- Three documents were converted into vector form using BoW, TF, and TF-IDF techniques and similarity was calculated using cosine similarity.
- In BoW vectorization, similarity is based on raw word counts, so documents sharing common words like *"machine"* and *"learning"* show higher similarity even if the meaning is not deeply related.
- In TF vectorization, normalization improves the comparison by considering the proportion of words instead of raw counts, resulting in slightly more balanced similarity values.
- In TF-IDF vectorization, rare and meaningful words are given higher importance while common words are reduced in weight, leading to more accurate and meaningful similarity results.
- It was observed that documents with similar context had higher similarity scores, while unrelated documents showed lower similarity across all methods.
- Among all three techniques, TF-IDF provided the best representation for measuring document similarity because it considers both term frequency and word importance across the corpus.

Post Lab Exercise:

Comment over the answer obtained in each of the case.

a. BoW Vectorization

- BoW measures similarity based on raw word counts.
- Documents with common words like *"machine"* and *"learning"* show high similarity.
- It does not consider importance of words.
- Common words dominate similarity scores.
- Less accurate for semantic comparison.

a. TF Vectorization

- TF normalizes word counts, improving comparison.
- Reduces bias due to longer sentences.
- Similarity is more balanced than BoW.
- Still gives importance to frequent words, even if they are not meaningful.
- Slight improvement over BoW.

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	**Date:**	**Enrolment No : 92301733009**

b. TF-IDF Vectorization

- TF-IDF gives higher weight to important (rare) words.
- Common words like *"I", "is"* get lower importance.
- Produces more meaningful similarity results.
- Best among all three methods for text similarity.
- Captures importance of words across documents.

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	**Date:**	**Enrolment No : 92301733009**

[4] ✓ 0s

```
documents = [
    "I love machine learning",

    "Machine learning is amazing",
    "I enjoy coding in python"
]
```

[5] ✓ 0s

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# BoW
bow = CountVectorizer()
X_bow = bow.fit_transform(documents)

# Similarity
bow_similarity = cosine_similarity(X_bow)

print("BoW Similarity Matrix:\n", bow_similarity)
```

```
BoW Similarity Matrix:
 [[1.         0.57735027 0.        ]
 [0.57735027 1.         0.        ]
 [0.         0.         1.        ]]
```

[6] ✓ 0s

```
from sklearn.feature_extraction.text import TfidfVectorizer

# TF (no IDF)
tf = TfidfVectorizer(use_idf=False, norm='l2')
X_tf = tf.fit_transform(documents)

# Similarity
tf_similarity = cosine_similarity(X_tf)

print("TF Similarity Matrix:\n", tf_similarity)
```

```
TF Similarity Matrix:
 [[1.         0.57735027 0.        ]
 [0.57735027 1.         0.        ]
 [0.         0.         1.        ]]
```

[7] ✓ 0s

```
# TF-IDF
tfidf = TfidfVectorizer()
X_tfidf = tfidf.fit_transform(documents)

# Similarity
tfidf_similarity = cosine_similarity(X_tfidf)

print("TF-IDF Similarity Matrix:\n", tfidf_similarity)
```

```
TF-IDF Similarity Matrix:
 [[1.         0.44333251 0.        ]
 [0.44333251 1.         0.        ]
 [0.         0.         1.        ]]
```